

JavaScript Objects (Part 1)

What is an Object?

Definition

An **Object** is a collection of related data stored as **key-value pairs**.

Objects allow you to store multiple related values in a single variable.

Why Do We Use Objects?

Objects are useful when you want to store information about one thing.

For example:

- User Information
- Product Details
- Student Data
- Employee Information
- Car Details

Instead of creating many variables:

```
let name = "Rokon";  
let age = 23;  
let country = "Bangladesh";
```

We can use one object.

```
const user = {  
  name: "Rokon",  
  age: 23,  
  country: "Bangladesh"  
};
```

Object Structure

```
const user = {  
  name: "Rokon",  
  age: 23,  
  country: "Bangladesh"  
};
```

Object

user

↓

{

name : "Rokon"

age : 23

country : "Bangladesh"

}

Here,

name

is called the **Key (Property)**

"Rokon"

is called the **Value**

Creating Objects

There are several ways to create objects.

Method 1 (Most Common)

```
const student = {  
  name: "Rokon",  
  age: 23,  
  cgpa: 3.85  
};  
  
console.log(student);
```

Output

```
{  
  name: "Rokon",  
  age: 23,  
  cgpa: 3.85  
}
```

Method 2

Create an empty object.

```
const person = {};  
  
person.name = "Rokon";  
person.age = 23;
```

```
console.log(person);
```

Output

```
{  
  name: "Rokon",  
  age: 23  
}
```

Properties

Definition

A **Property** is a key-value pair inside an object.

Example

```
const laptop = {  
  
  brand: "HP",  
  
  model: "EliteBook",  
  
  ram: "16GB"  
  
};
```

Properties

Property	Value
brand	HP
model	EliteBook

Property	Value
ram	16GB

Another Example

```
const car = {  
  brand: "Toyota",  
  color: "Black",  
  price: 25000  
};
```

Accessing Properties

There are two ways.

1. Dot Notation (Recommended)

```
const student = {  
  name: "Rokon",  
  age: 23  
};  
  
console.log(student.name);  
console.log(student.age);
```

Output

```
Rokon  
23
```

2. Bracket Notation

```
console.log(student["name"]);  
  
console.log(student["age"]);
```

Output

```
Rokon  
23
```

When to Use Bracket Notation?

Useful when the property name is stored inside a variable.

```
const student = {  
  name: "Rokon",  
  age: 23  
};  
  
let key = "name";  
  
console.log(student[key]);
```

Output

Rokon

Updating Properties

Definition

You can change the value of an existing property.

Example

```
const student = {  
  name: "Rokon",  
  age: 23  
};  
  
student.age = 24;  
  
console.log(student);
```

Output

```
{  
  name: "Rokon",  
  age: 24  
}
```

Update Multiple Properties

```
const product = {  
  name: "Laptop",  
  price: 50000  
};  
  
product.price = 45000;  
  
product.name = "Gaming Laptop";  
  
console.log(product);
```

Output

```
{  
  name: "Gaming Laptop",  
  price: 45000  
}
```

Adding Properties

Definition

New properties can be added at any time.

Example

```
const student = {  
  name: "Rokon"  
};  
  
student.age = 23;
```



```
student.country = "Bangladesh";  
  
console.log(student);
```

Output

```
{  
  name: "Rokon",  
  age: 23,  
  country: "Bangladesh"  
}
```

Real Project Example

```
const user = {  
  name: "John"  
};  
  
user.email = "john@gmail.com";  
user.role = "Admin";  
  
console.log(user);
```

Deleting Properties

Definition

The **delete** keyword removes a property from an object.

Syntax

```
delete object.property;
```

Example

```
const student = {  
  name: "Rokon",  
  age: 23,  
  cgpa: 3.90  
};  
  
delete student.cgpa;  
  
console.log(student);
```

Output

```
{  
  name: "Rokon",  
  age: 23  
}
```

Another Example

```
const car = {  
  brand: "Toyota",  
  color: "Black",  
  price: 25000  
}
```

```
};

delete car.price;

console.log(car);
```

Output

```
{
  brand: "Toyota",
  color: "Black"
}
```

CRUD Operations with Objects

Operation	Description
Create	Create a new object
Read	Access properties
Update	Change property values
Delete	Remove properties

Example

```
const student = {

  name: "Rokon"

};

// Create
student.age = 23;

// Read
console.log(student.name);

// Update
```

```
student.age = 24;

// Delete
delete student.name;

console.log(student);
```

Real Project Example

User Profile

```
const user = {

  name: "John",

  email: "john@gmail.com",

  isVerified: true

};

console.log(user.name);

user.isVerified = false;

user.phone = "01712345678";

delete user.email;

console.log(user);
```

Best Practices

Use **const** when creating objects.

```
const user = {};
```

Use **dot notation** whenever possible.

```
user.name
```

Use meaningful property names.

```
firstName
```

```
lastName
```

```
email
```

```
phone
```

Common Mistakes

Accessing Wrong Property

```
const user = {  
  name: "Rokon"  
};  
  
console.log(user.age);
```

Output

```
undefined
```

Forgetting Quotes in Bracket Notation

```
user[name]
```

```
user["name"]
```

Deleting a Non-Existing Property

```
delete user.salary;
```

No error occurs, but nothing is deleted.

Interview Questions

What is an Object?

An object is a collection of key-value pairs used to store related data.

What is a Property?

A property is a key-value pair inside an object.

How do you access object properties?

- Dot Notation

```
user.name
```

- Bracket Notation

```
user["name"]
```

How do you update a property?

```
user.age = 24;
```

How do you add a property?

```
user.country = "Bangladesh";
```

How do you delete a property?

```
delete user.age;
```

Summary Table

Topic	Description
Object	Collection of key-value pairs
Property	Key-value pair inside an object
Create	<code>{}</code>
Access	Dot / Bracket notation
Update	<code>object.property = value</code>
Add	<code>object.newProperty = value</code>
Delete	<code>delete object.property</code>

Topic	Description
CRUD	Create, Read, Update, Delete

JavaScript Objects (Part 2A)

This section covers:

- Object Methods
- this Keyword
- Real Project Examples
- Common Mistakes
- Interview Questions

Object Methods

Definition

An **Object Method** is a function stored inside an object.

Methods allow an object to perform actions.

Unlike properties, methods contain executable code.

Object Structure


```
const person = {  
  
  name: "Rokon",  
  
  age: 23,  
  
  greet: function () {  
  
    console.log("Hello!");  
  
  }  
  
};
```

Here,

Item	Type
name	Property
age	Property
greet	Method

Creating an Object Method

Syntax

```
const object = {  
  
  methodName: function(){  
  
    // code  
  
  }  
  
};
```

Example

```
const student = {  
  name: "Rokon",  
  greet: function(){  
    console.log("Welcome!");  
  }  
};  
  
student.greet();
```

Output

```
Welcome!
```

ES6 Short Method Syntax

JavaScript provides a shorter syntax for methods.

Instead of writing:

```
const user = {  
  greet: function(){  
    console.log("Hello");  
  }  
};
```

Write

```
const user = {  
  greet(){  
    console.log("Hello");  
  }  
};
```

Output

```
Hello
```

This is the preferred syntax in modern JavaScript.

Methods Can Use Properties

```
const student = {  
  name: "Rokon",  
  greet(){  
    console.log("Hello " + this.name);  
  }  
};  
  
student.greet();
```

Output

```
Hello Rokon
```

Returning Values

Methods can return values.

```
const calculator = {  
  add(){  
    return 20 + 30;  
  }  
};  
  
console.log(calculator.add());
```

Output

```
50
```

Multiple Methods

```
const calculator = {  
  add(a,b){  
    return a+b;  
  },  
  subtract(a,b){  
    return a-b;  
  }  
};  
  
console.log(calculator.add(20,10));
```

```
console.log(calculator.subtract(20,10));
```

Output

```
30
```

```
10
```

Why Use Methods?

Methods group behavior with data.

Example

Instead of

```
let name = "Rokon";

function greet(){

    console.log("Hello");

}
```

Use

```
const user = {  
  name: "Rokon",  
  greet(){  
    console.log("Hello");  
  }  
};
```

Everything related to the user stays inside one object.

The this Keyword

Definition

The **this** keyword refers to the object that is calling the method.

Example

```
const student = {  
  name: "Rokon",  
  age: 23,  
  introduce(){  
    console.log(this.name);  
  }  
};  
  
student.introduce();
```

Output

Rokon

Explanation

```
student
↓
introduce()
↓
this
↓
student
```

Therefore,

```
this.name
```

means

```
student.name
```

Access Multiple Properties

```
const student = {
  name: "Rokon",
```

```
    age: 23,  
  
    cgpa: 3.90,  
  
    details(){  
  
        console.log(this.name);  
  
        console.log(this.age);  
  
        console.log(this.cgpa);  
  
    }  
  
};  
  
student.details();
```

Output

```
Rokon  
  
23  
  
3.9
```

Without this

```
const student = {  
  name: "Rokon",  
  greet(){  
    console.log(name);  
  }  
};  
  
student.greet();
```

Output

```
ReferenceError
```

Because JavaScript searches for a variable named `name`, not the object's property.

Correct

```
this.name
```

this in Another Object

```
const car = {  
  brand: "Toyota",  
  model: "Corolla",  
  show(){  
    console.log(this.brand);  
    console.log(this.model);  
  }  
};
```

```
    }  
  
};  
  
car.show();
```

Output

```
Toyota  
  
Corolla
```

Real Project Example 1

User Profile

```
const user = {  
  
  firstName: "MD",  
  
  lastName: "Rokon",  
  
  fullName(){  
  
    return this.firstName + " " + this.lastName;  
  
  }  
  
};  
  
console.log(user.fullName());
```

Output

```
MD Rokon
```

Real Project Example 2

Shopping Cart

```
const product = {  
  name: "Laptop",  
  price: 50000,  
  discount: 5000,  
  finalPrice(){  
    return this.price - this.discount;  
  }  
};  
  
console.log(product.finalPrice());
```

Output

```
45000
```

Real Project Example 3

Bank Account

```
const account = {  
  owner: "Rokon",  
  balance: 10000,  
  deposit(amount){
```

```
        this.balance += amount;

    }

};

account.deposit(5000);

console.log(account.balance);
```

Output

```
15000
```

Real Project Example 4

Student Result

```
const student = {

    name: "Rokon",

    marks: [80,90,95],

    average(){

        return (80+90+95)/3;

    }

};

console.log(student.average());
```

Output

```
88.33333333333333
```

Best Practices

Use ES6 method syntax.

```
greet(){  
  
}
```

Use **this** to access properties inside methods.

```
this.name
```

Keep related data and functions inside one object.

Use meaningful method names.

Examples

```
login()  
  
logout()  
  
calculateTotal()  
  
getFullName()  
  
addToCart()
```

Common Mistakes

Forgetting Parentheses

Wrong

```
console.log(user.greet);
```

Output

```
[Function]
```

Correct

```
console.log(user.greet());
```

Forgetting this

Wrong

```
console.log(name);
```

Correct

```
console.log(this.name);
```

Using Arrow Functions as Object Methods

Wrong

```
const user = {
```

```
    name: "Rokon",

    greet: () => {

        console.log(this.name);

    }

};
```

this inside an arrow function does **not** refer to the object.

Correct

```
const user = {

    name: "Rokon",

    greet(){

        console.log(this.name);

    }

};
```

Typo in Property Names

Wrong

```
this.fullname
```

Correct

```
this.fullName
```

JavaScript property names are case-sensitive.

Interview Questions

What is an Object Method?

A method is a function stored inside an object.

What is the difference between a Property and a Method?

Property	Method
Stores data	Stores a function
Example: <code>name</code>	Example: <code>greet()</code>

What does `this` refer to?

`this` refers to the object that calls the method.

Why should you use `this`?

It allows a method to access the object's own properties.

Can an object have multiple methods?

Yes.

```
const calculator = {  
  add(){},  
  subtract(){},  
  multiply(){},  
  divide(){}  
};
```


Which syntax is preferred for object methods?

```
greet(){  
  
}
```

instead of

```
greet: function(){  
  
}
```

Can methods return values?

Yes.

```
square(n){  
  
    return n*n;  
  
}
```

Summary Table

Topic	Description
Method	Function inside an object
ES6 Method Syntax	<code>greet(){}</code>
this	Refers to the current object
Access Property	<code>this.property</code>
Return Value	<code>return</code> statement
Call Method	<code>object.method()</code>

Topic	Description
Best Practice	Use ES6 methods with this